alu_pipelined.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (4 of 18) - GVIM9 (on

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | **alu_pipelined.sv** | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | compare_unit.sv | divide_

```
//------------------------------------------------------------------------------
// Module: alu_pipelined
// Description: 32-bit Pipelined ALU with Comprehensive Status Flags
//              3-stage pipeline with operand forwarding
//              Supports 32 operations across 8 functional categories
//------------------------------------------------------------------------------

module alu_pipelined
  import alu_pkg::*;
#(
  parameter int DATA_WIDTH = 32,
  parameter int PIPELINE_STAGES = 3
)(
  // Clock and Reset
  input  logic                    clk,
  input  logic                    rst_n,

  // Input Interface (Valid/Ready handshake)
  input  logic                    in_valid,
  output logic                    in_ready,
  input  logic [DATA_WIDTH-1:0]   operand_a,
  input  logic [DATA_WIDTH-1:0]   operand_b,
  input  logic [OPCODE_WIDTH-1:0] opcode,

  // Output Interface (Valid/Ready handshake)
  output logic                    out_valid,
  input  logic                    out_ready,
  output logic [DATA_WIDTH-1:0]   result,

  // Status Flags
  output logic                    zero,
  output logic                    carry,
  output logic                    overflow,
  output logic                    negative,
  output logic                    parity,
  output logic                    saturation
);

  //============================================================================
  // Internal Signals
  //============================================================================

  // Pipeline registers
  stage1_reg_t stage1_reg, stage1_next;
  stage2_reg_t stage2_reg, stage2_next;

  // Pipeline control
  logic stall;
  logic pipeline_advance;
```

alu_pipelined.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (4 of 18) - GVIM9 (on ip-10-115-24

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | **alu_pipelined.sv** | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | compare_unit.sv | divide_unit.sv

```
stage1_reg_t stage1_reg, stage1_next;
stage2_reg_t stage2_reg, stage2_next;

// Pipeline control
logic stall;
logic pipeline_advance;

// Forwarding signals
fwd_sel_t forward_a_sel, forward_b_sel;
logic [DATA_WIDTH-1:0] operand_a_fwd, operand_b_fwd;

// Execute stage signals
logic [DATA_WIDTH-1:0] exec_result;
logic [FLAG_WIDTH-1:0] exec_flags;

// Carry register for ADDC/SUBB operations
logic carry_reg;

//============================================================================
// Pipeline Control Logic
//============================================================================

// Stall when output is valid but consumer not ready
assign stall = stage2_reg.valid & ~out_ready;

// Pipeline advances when not stalled
assign pipeline_advance = ~stall;

// Input ready when pipeline can accept new data
assign in_ready = pipeline_advance;

//============================================================================
// Stage 1: Fetch / Decode / Hazard Detection
//============================================================================

// Hazard Detection and Forwarding Select for Operand A
always_comb begin
  // Default: no forwarding
  forward_a_sel = FWD_NONE;

  // Check Stage 2 first (higher priority - more recent result)
  if (stage1_reg.valid) begin
    forward_a_sel = FWD_STAGE2;
  end
  // Check Stage 3
  else if (stage2_reg.valid) begin
    forward_a_sel = FWD_STAGE3;
  end
end
```

alu_pipelined.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (4 of 18) - GVIM

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | **alu_pipelined.sv** | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | compare_unit.sv

```
// Hazard Detection and Forwarding Select for Operand B
always_comb begin
  // Default: no forwarding
  forward_b_sel = FWD_NONE;

  // Check Stage 2 first (higher priority - more recent result)
  if (stage1_reg.valid) begin
    forward_b_sel = FWD_STAGE2;
  end
  // Check Stage 3
  else if (stage2_reg.valid) begin
    forward_b_sel = FWD_STAGE3;
  end
end

// Forwarding Mux for Operand A
always_comb begin
  case (forward_a_sel)
    FWD_STAGE2: operand_a_fwd = exec_result;
    FWD_STAGE3: operand_a_fwd = stage2_reg.result;
    default:    operand_a_fwd = stage1_reg.operand_a;
  endcase
end

// Forwarding Mux for Operand B
always_comb begin
  case (forward_b_sel)
    FWD_STAGE2: operand_b_fwd = exec_result;
    FWD_STAGE3: operand_b_fwd = stage2_reg.result;
    default:    operand_b_fwd = stage1_reg.operand_b;
  endcase
end

// Stage 1 Register Update
always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    stage1_reg <= '0;
  end else if (pipeline_advance) begin
    stage1_reg.operand_a <= operand_a;
    stage1_reg.operand_b <= operand_b;
    stage1_reg.opcode    <= opcode;
    stage1_reg.carry_in  <= carry_reg;
    stage1_reg.valid     <= in_valid & in_ready;
  end
end

//============================================================================
// Stage 2: Execute
//============================================================================
```

Applications Places System

alu_pipelined.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (4 of 18) - GV

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | **alu_pipelined.sv** | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | compare_unit.sv

```
//==============================================================================
// Stage 2: Execute
//==============================================================================

// Functional unit selection
logic [2:0] func_sel;
assign func_sel = stage1_reg.opcode[4:2];

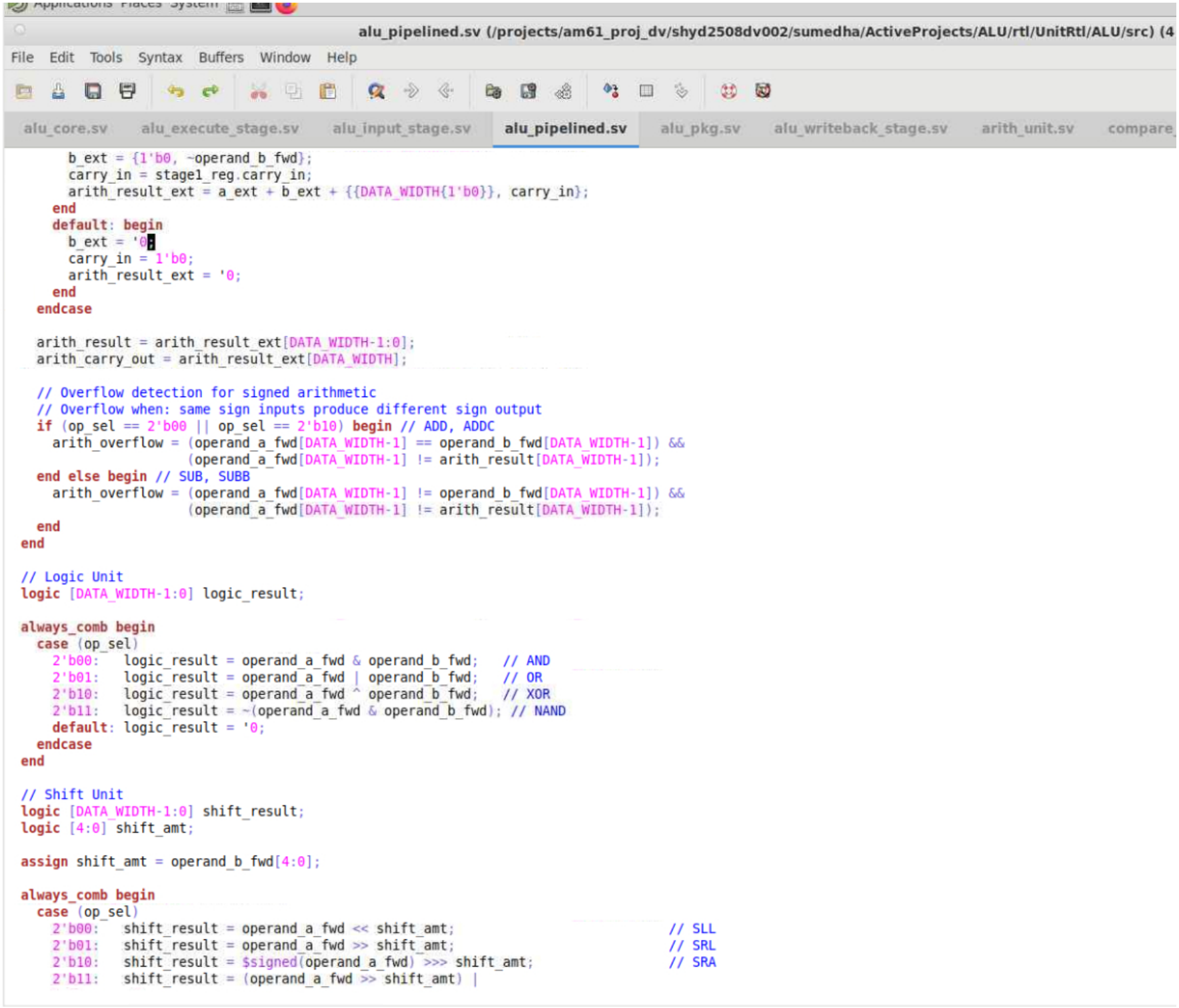
// Operation selection within unit
logic [1:0] op_sel;
assign op_sel = stage1_reg.opcode[1:0];

// Arithmetic Unit Signals
logic [DATA_WIDTH:0]   arith_result_ext;  // 33-bit for carry
logic [DATA_WIDTH-1:0] arith_result;
logic                  arith_carry_out;
logic                  arith_overflow;

// Arithmetic Unit
always_comb begin
  logic [DATA_WIDTH:0] a_ext, b_ext;
  logic carry_in;

  a_ext = {1'b0, operand_a_fwd};

  case (op_sel)
    2'b00: begin // ADD
      b_ext = {1'b0, operand_b_fwd};
      carry_in = 1'b0;
      arith_result_ext = a_ext + b_ext + {{DATA_WIDTH{1'b0}}, carry_in};
    end
    2'b01: begin // SUB
      b_ext = {1'b0, ~operand_b_fwd};
      carry_in = 1'b1;
      arith_result_ext = a_ext + b_ext + {{DATA_WIDTH{1'b0}}, carry_in};
    end
    2'b10: begin // ADDC
      b_ext = {1'b0, operand_b_fwd};
      carry_in = stage1_reg.carry_in;
      arith_result_ext = a_ext + b_ext + {{DATA_WIDTH{1'b0}}, carry_in};
    end
    2'b11: begin // SUBB
      b_ext = {1'b0, ~operand_b_fwd};
      carry_in = stage1_reg.carry_in;
      arith_result_ext = a_ext + b_ext + {{DATA_WIDTH{1'b0}}, carry_in};
    end
    default: begin
      b_ext = '0;
      carry_in = 1'b0;
      arith_result_ext = '0;
```


alu_pipelined.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (4
File Edit Tools Syntax Buffers Window Help
alu_core.sv
alu_execute_stage.sv
alu_input_stage.sv
alu_pipelined.sv
alu_pkg.sv
alu_writeback_stage.sv
arith_unit.sv
compare_
b_ext = {1'b0, ~operand_b_fwd};
carry_in = stage1_reg.carry_in;
arith_result_ext = a_ext + b_ext + {{DATA_WIDTH{1'b0}}, carry_in};
end
default: begin
b_ext = '0;
carry_in = 1'b0;
arith_result_ext = '0;
end
endcase
arith_result = arith_result_ext[DATA_WIDTH-1:0];
arith_carry_out = arith_result_ext[DATA_WIDTH];
// Overflow detection for signed arithmetic
// Overflow when: same sign inputs produce different sign output
if (op_sel == 2'b00 || op_sel == 2'b10) begin // ADD, ADDC
arith_overflow = (operand_a_fwd[DATA_WIDTH-1] == operand_b_fwd[DATA_WIDTH-1]) &&
(operand_a_fwd[DATA_WIDTH-1] != arith_result[DATA_WIDTH-1]);
end else begin // SUB, SUBB
arith_overflow = (operand_a_fwd[DATA_WIDTH-1] != operand_b_fwd[DATA_WIDTH-1]) &&
(operand_a_fwd[DATA_WIDTH-1] != arith_result[DATA_WIDTH-1]);
end
end
// Logic Unit
logic [DATA_WIDTH-1:0] logic_result;
always_comb begin
case (op_sel)
2'b00: logic_result = operand_a_fwd & operand_b_fwd; // AND
2'b01: logic_result = operand_a_fwd | operand_b_fwd; // OR
2'b10: logic_result = operand_a_fwd ^ operand_b_fwd; // XOR
2'b11: logic_result = ~(operand_a_fwd & operand_b_fwd); // NAND
default: logic_result = '0;
endcase
end
// Shift Unit
logic [DATA_WIDTH-1:0] shift_result;
logic [4:0] shift_amt;
assign shift_amt = operand_b_fwd[4:0];
always_comb begin
case (op_sel)
2'b00: shift_result = operand_a_fwd << shift_amt; // SLL
2'b01: shift_result = operand_a_fwd >> shift_amt; // SRL
2'b10: shift_result = $signed(operand_a_fwd) >>> shift_amt; // SRA
2'b11: shift_result = (operand_a_fwd >> shift_amt) |

alu_pipelined.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (4 of

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | **alu_pipelined.sv** | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | compare_un

```
assign shift_amt = operand_b_fwd[4:0];

always_comb begin
  case (op_sel)
    2'b00:   shift_result = operand_a_fwd << shift_amt;                          // SLL
    2'b01:   shift_result = operand_a_fwd >> shift_amt;                          // SRL
    2'b10:   shift_result = $signed(operand_a_fwd) >>> shift_amt;                // SRA
    2'b11:   shift_result = (operand_a_fwd >> shift_amt) |
                            (operand_a_fwd << (DATA_WIDTH - shift_amt));         // ROR
    default: shift_result = '0;
  endcase
end

// Compare Unit
logic [DATA_WIDTH-1:0] compare_result;
logic signed [DATA_WIDTH-1:0] signed_a, signed_b;

assign signed_a = $signed(operand_a_fwd);
assign signed_b = $signed(operand_b_fwd);

always_comb begin
  case (op_sel)
    2'b00:   compare_result = (signed_a < signed_b) ? 32'd1 : 32'd0;             // SLT
    2'b01:   compare_result = (operand_a_fwd < operand_b_fwd) ? 32'd1 : 32'd0;   // SLTU
    2'b10:   compare_result = (signed_a > signed_b) ? 32'd1 : 32'd0;             // SGT
    2'b11:   compare_result = (operand_a_fwd == operand_b_fwd) ? 32'd1 : 32'd0;  // SEQ
    default: compare_result = '0;
  endcase
end

// Multiply Unit
logic [DATA_WIDTH-1:0] mul_result;
logic signed [2*DATA_WIDTH-1:0] mul_full_signed;
logic [2*DATA_WIDTH-1:0] mul_full_unsigned;
logic signed [2*DATA_WIDTH-1:0] mul_full_mixed;

always_comb begin
  mul_full_signed   = $signed(operand_a_fwd) * $signed(operand_b_fwd);
  mul_full_unsigned = operand_a_fwd * operand_b_fwd;
  mul_full_mixed    = $signed(operand_a_fwd) * $signed({1'b0, operand_b_fwd});

  case (op_sel)
    2'b00:   mul_result = mul_full_unsigned[DATA_WIDTH-1:0];                     // MUL
    2'b01:   mul_result = mul_full_signed[2*DATA_WIDTH-1:DATA_WIDTH];            // MULH
    2'b10:   mul_result = mul_full_unsigned[2*DATA_WIDTH-1:DATA_WIDTH];          // MULHU
    2'b11:   mul_result = mul_full_mixed[2*DATA_WIDTH-1:DATA_WIDTH];             // MULSU
    default: mul_result = '0;
  endcase
end
```

alu_pipelined.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src)

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | **alu_pipelined.sv** | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | compa

```
    default: mul_result = '0;
  endcase
end

// Divide Unit
logic [DATA_WIDTH-1:0] div_result;
logic signed [DATA_WIDTH-1:0] div_signed_a, div_signed_b;
logic [DATA_WIDTH-1:0] quotient_signed, quotient_unsigned;
logic [DATA_WIDTH-1:0] remainder_signed, remainder_unsigned;

assign div_signed_a = $signed(operand_a_fwd);
assign div_signed_b = $signed(operand_b_fwd);

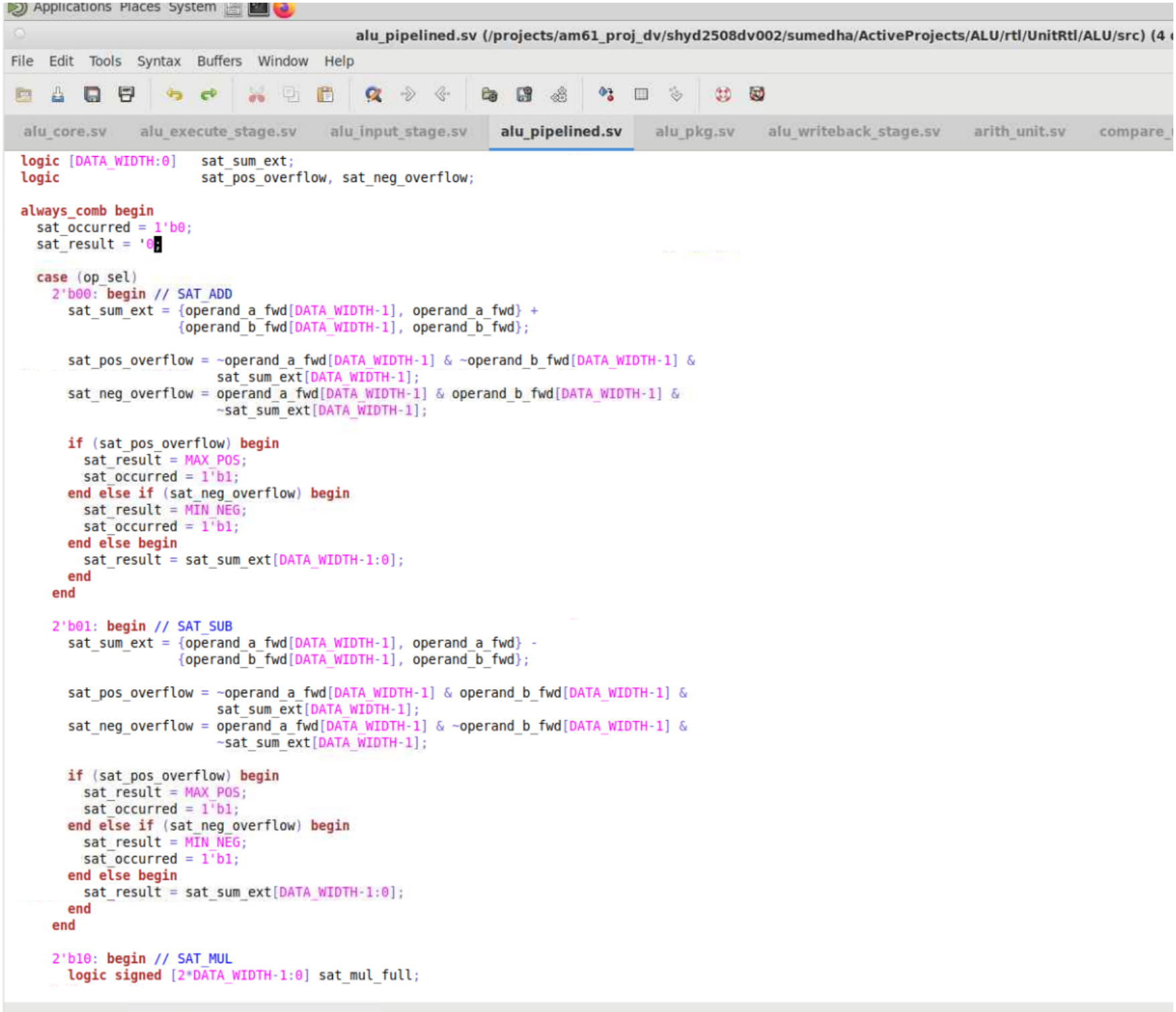
always_comb begin
  // Handle divide by zero
  if (operand_b_fwd == '0) begin
    quotient_signed   = '1;  // All 1s for divide by zero
    quotient_unsigned = '1;
    remainder_signed  = operand_a_fwd;
    remainder_unsigned = operand_a_fwd;
  end else begin
    quotient_signed   = div_signed_a / div_signed_b;
    quotient_unsigned = operand_a_fwd / operand_b_fwd;
    remainder_signed  = div_signed_a % div_signed_b;
    remainder_unsigned = operand_a_fwd % operand_b_fwd;
  end

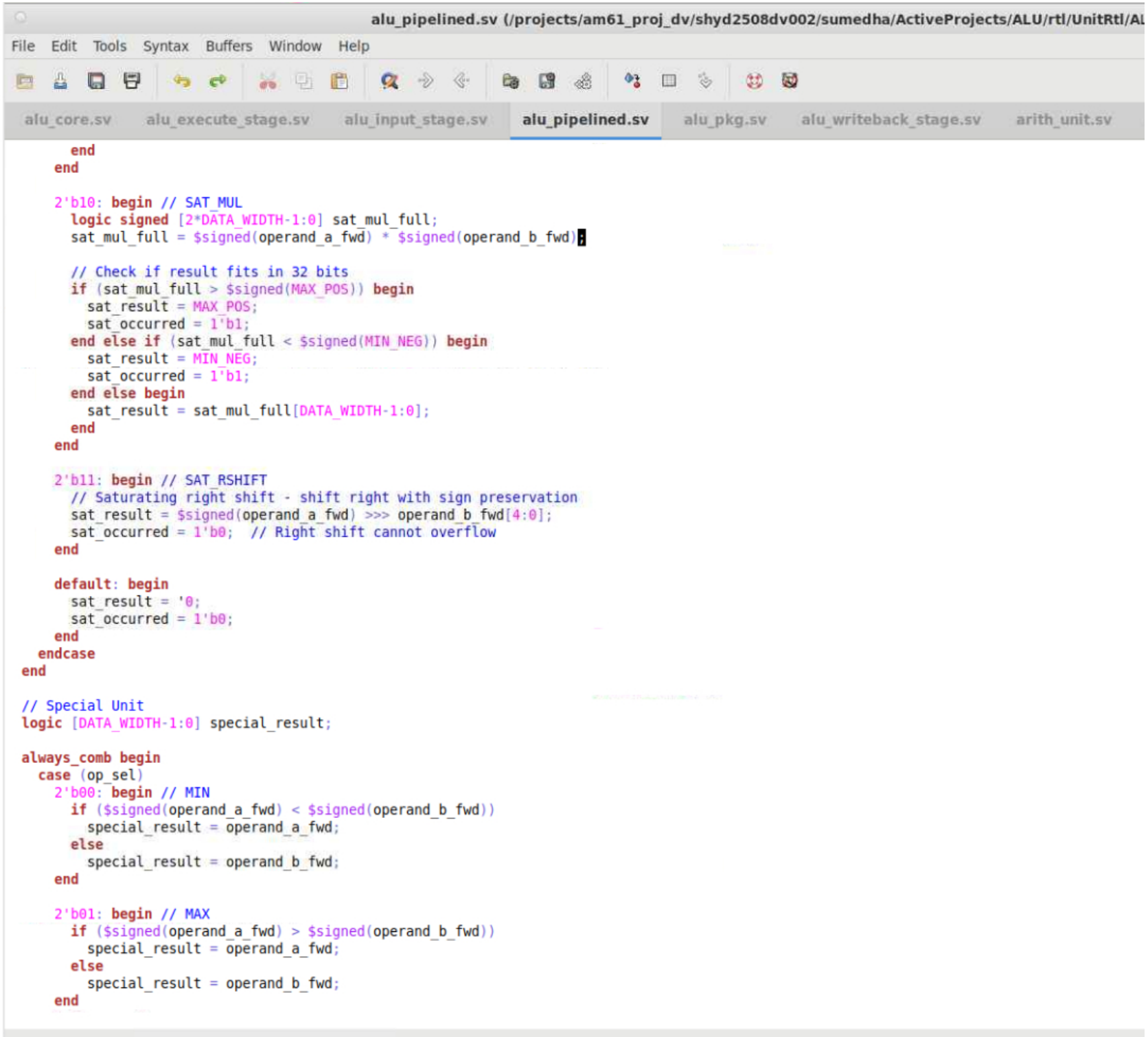
  case (op_sel)
    2'b00:   div_result = quotient_signed;    // DIV
    2'b01:   div_result = quotient_unsigned;  // DIVU
    2'b10:   div_result = remainder_signed;   // REM
    2'b11:   div_result = remainder_unsigned; // REMU
    default: div_result = '0;
  endcase
end

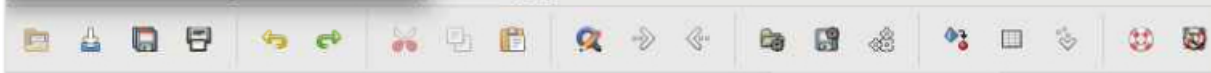
// Saturation Unit
logic [DATA_WIDTH-1:0] sat_result;
logic                  sat_occurred;
logic [DATA_WIDTH:0]   sat_sum_ext;
logic                  sat_pos_overflow, sat_neg_overflow;

always_comb begin
  sat_occurred = 1'b0;
  sat_result = '0;

  case (op_sel)
    2'b00: begin // SAT_ADD
      sat_sum_ext = {operand_a_fwd[DATA_WIDTH-1], operand_a_fwd} +
                    {operand_b_fwd[DATA_WIDTH-1], operand_b_fwd};
```


Applications Places System
alu_pipelined.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (4
File Edit Tools Syntax Buffers Window Help
alu_core.sv
alu_execute_stage.sv
alu_input_stage.sv
alu_pipelined.sv
alu_pkg.sv
alu_writeback_stage.sv
arith_unit.sv
compare_
logic [DATA_WIDTH:0] sat_sum_ext;
logic sat_pos_overflow, sat_neg_overflow;
always_comb begin
sat_occurred = 1'b0;
sat_result = '0;
case (op_sel)
2'b00: begin // SAT_ADD
sat_sum_ext = {operand_a_fwd[DATA_WIDTH-1], operand_a_fwd} +
{operand_b_fwd[DATA_WIDTH-1], operand_b_fwd};
sat_pos_overflow = ~operand_a_fwd[DATA_WIDTH-1] & ~operand_b_fwd[DATA_WIDTH-1] &
sat_sum_ext[DATA_WIDTH-1];
sat_neg_overflow = operand_a_fwd[DATA_WIDTH-1] & operand_b_fwd[DATA_WIDTH-1] &
~sat_sum_ext[DATA_WIDTH-1];
if (sat_pos_overflow) begin
sat_result = MAX_POS;
sat_occurred = 1'b1;
end else if (sat_neg_overflow) begin
sat_result = MIN_NEG;
sat_occurred = 1'b1;
end else begin
sat_result = sat_sum_ext[DATA_WIDTH-1:0];
end
end
2'b01: begin // SAT_SUB
sat_sum_ext = {operand_a_fwd[DATA_WIDTH-1], operand_a_fwd} -
{operand_b_fwd[DATA_WIDTH-1], operand_b_fwd};
sat_pos_overflow = ~operand_a_fwd[DATA_WIDTH-1] & operand_b_fwd[DATA_WIDTH-1] &
sat_sum_ext[DATA_WIDTH-1];
sat_neg_overflow = operand_a_fwd[DATA_WIDTH-1] & ~operand_b_fwd[DATA_WIDTH-1] &
~sat_sum_ext[DATA_WIDTH-1];
if (sat_pos_overflow) begin
sat_result = MAX_POS;
sat_occurred = 1'b1;
end else if (sat_neg_overflow) begin
sat_result = MIN_NEG;
sat_occurred = 1'b1;
end else begin
sat_result = sat_sum_ext[DATA_WIDTH-1:0];
end
end
2'b10: begin // SAT_MUL
logic signed [2*DATA_WIDTH-1:0] sat_mul_full;


alu_pipelined.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/AL
File Edit Tools Syntax Buffers Window Help
alu_core.sv alu_execute_stage.sv alu_input_stage.sv alu_pipelined.sv alu_pkg.sv alu_writeback_stage.sv arith_unit.sv
end
end
2'b10: begin // SAT_MUL
logic signed [2*DATA_WIDTH-1:0] sat_mul_full;
sat_mul_full = $signed(operand_a_fwd) * $signed(operand_b_fwd);
// Check if result fits in 32 bits
if (sat_mul_full > $signed(MAX_POS)) begin
sat_result = MAX_POS;
sat_occurred = 1'b1;
end else if (sat_mul_full < $signed(MIN_NEG)) begin
sat_result = MIN_NEG;
sat_occurred = 1'b1;
end else begin
sat_result = sat_mul_full[DATA_WIDTH-1:0];
end
end
2'b11: begin // SAT_RSHIFT
// Saturating right shift - shift right with sign preservation
sat_result = $signed(operand_a_fwd) >>> operand_b_fwd[4:0];
sat_occurred = 1'b0; // Right shift cannot overflow
end
default: begin
sat_result = '0;
sat_occurred = 1'b0;
end
endcase
end
// Special Unit
logic [DATA_WIDTH-1:0] special_result;
always_comb begin
case (op_sel)
2'b00: begin // MIN
if ($signed(operand_a_fwd) < $signed(operand_b_fwd))
special_result = operand_a_fwd;
else
special_result = operand_b_fwd;
end
2'b01: begin // MAX
if ($signed(operand_a_fwd) > $signed(operand_b_fwd))
special_result = operand_a_fwd;
else
special_result = operand_b_fwd;
end




```
always_comb begin
  case (op_sel)
    2'b00: begin // MIN
      if ($signed(operand_a_fwd) < $signed(operand_b_fwd))
        special_result = operand_a_fwd;
      else
        special_result = operand_b_fwd;
    end

    2'b01: begin // MAX
      if ($signed(operand_a_fwd) > $signed(operand_b_fwd))
        special_result = operand_a_fwd;
      else
        special_result = operand_b_fwd;
    end

    2'b10: begin // ABS
      if (operand_a_fwd[DATA_WIDTH-1])
        special_result = (~operand_a_fwd) + 1'b1;
      else
        special_result = operand_a_fwd;
    end

    2'b11: begin // CLZ
      special_result = {26'b0, count_leading_zeros(operand_a_fwd)};
    end

    default: special_result = '0;
  endcase
end

// Result Multiplexer
always_comb begin
  case (func_sel)
    FUNC_ARITH:    exec_result = arith_result;
    FUNC_LOGIC:    exec_result = logic_result;
    FUNC_SHIFT:    exec_result = shift_result;
    FUNC_COMPARE:  exec_result = compare_result;
    FUNC_MULTIPLY: exec_result = mul_result;
    FUNC_DIVIDE:   exec_result = div_result;
    FUNC_SATURATE: exec_result = sat_result;
    FUNC_SPECIAL:  exec_result = special_result;
    default:       exec_result = '0;
  endcase
end

// Flag Generation
logic flag_zero, flag_carry, flag_overflow, flag_negative, flag_parity, flag_saturation;
```

alu_pipelined.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (4 o

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | **alu_pipelined.sv** | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | compare_u

```
logic flag_zero, flag_carry, flag_overflow, flag_negative, flag_parity, flag_saturation;

always_comb begin
  // Zero flag: result is all zeros
  flag_zero = (exec_result == '0);

  // Negative flag: MSB of result
  flag_negative = exec_result[DATA_WIDTH-1];

  // Parity flag: XOR reduction (even parity)
  flag_parity = ^exec_result;

  // Carry flag: only valid for arithmetic operations
  if (func_sel == FUNC_ARITH) begin
    flag_carry = arith_carry_out;
  end else begin
    flag_carry = 1'b0;
  end

  // Overflow flag: only valid for arithmetic (non-saturating)
  if (func_sel == FUNC_ARITH) begin
    flag_overflow = arith_overflow;
  end else begin
    flag_overflow = 1'b0;
  end

  // Saturation flag: only valid for saturation operations
  if (func_sel == FUNC_SATURATE) begin
    flag_saturation = sat_occurred;
  end else begin
    flag_saturation = 1'b0;
  end

  // Pack flags: {zero, carry, overflow, negative, parity, saturation}
  exec_flags = {flag_zero, flag_carry, flag_overflow, flag_negative, flag_parity, flag_saturation};
end

// Stage 2 Register Update
always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    stage2_reg <= '0;
  end else if (pipeline_advance) begin
    stage2_reg.result <= exec_result;
    stage2_reg.flags  <= exec_flags;
    stage2_reg.valid  <= stage1_reg.valid;
  end
end

// Update carry register for ADDC/SUBB
always_ff @(posedge clk or negedge rst_n) begin
```

alu_pipelined.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | **alu_pipelined.sv** | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | comp

```
    // Pack flags: {zero, carry, overflow, negative, parity, saturation}
    exec_flags = {flag_zero, flag_carry, flag_overflow, flag_negative, flag_parity, flag_saturation};
  end

  // Stage 2 Register Update
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      stage2_reg <= '0;
    end else if (pipeline_advance) begin
      stage2_reg.result <= exec_result;
      stage2_reg.flags  <= exec_flags;
      stage2_reg.valid  <= stage1_reg.valid;
    end
  end

  // Update carry register for ADDC/SUBB
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      carry_reg <= 1'b0;
    end else if (pipeline_advance && stage1_reg.valid && func_sel == FUNC_ARITH) begin
      carry_reg <= arith_carry_out;
    end
  end

  //============================================================================
  // Stage 3: Writeback / Output
  //============================================================================

  // Output assignments
  assign out_valid  = stage2_reg.valid;
  assign result     = stage2_reg.result;

  // Flag outputs (unpacked from flags vector)
  assign zero       = stage2_reg.flags[FLAG_ZERO];
  assign carry      = stage2_reg.flags[FLAG_CARRY];
  assign overflow   = stage2_reg.flags[FLAG_OVERFLOW];
  assign negative   = stage2_reg.flags[FLAG_NEGATIVE];
  assign parity     = stage2_reg.flags[FLAG_PARITY];
  assign saturation = stage2_reg.flags[FLAG_SATURATION];

endmodule : alu_pipelined
~
~
~
```

alu_pkg.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (5 of 18)

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | alu_pipelined.sv | **alu_pkg.sv** | alu_writeback_stage.sv | arith_unit.sv | compare_unit

```
//------------------------------------------------------------------------------
// Module: alu_pkg
// Description: Package containing types, parameters, and opcodes for the
//              32-bit Pipelined ALU
//------------------------------------------------------------------------------

package alu_pkg;

  //============================================================================
  // Parameters
  //============================================================================
  parameter int DATA_WIDTH      = 32;
  parameter int OPCODE_WIDTH    = 5;
  parameter int FLAG_WIDTH      = 6;
  parameter int PIPELINE_STAGES = 3;
  parameter int REG_ADDR_WIDTH  = 5;   // 5 bits for 32 registers

  // Saturation boundary values
  parameter logic [DATA_WIDTH-1:0] MAX_POS = 32'h7FFF_FFFF;
  parameter logic [DATA_WIDTH-1:0] MIN_NEG = 32'h8000_0000;

  //============================================================================
  // Opcode Encoding
  // Upper 3 bits [4:2] select functional unit
  // Lower 2 bits [1:0] select operation within unit
  //============================================================================

  // Arithmetic Operations (000xx)
  typedef enum logic [OPCODE_WIDTH-1:0] {
    OP_ADD      = 5'b00000,  // Addition (A + B)
    OP_SUB      = 5'b00001,  // Subtraction (A - B)
    OP_ADDC     = 5'b00010,  // Add with carry
    OP_SUBB     = 5'b00011,  // Subtract with borrow

    // Logical Operations (001xx)
    OP_AND      = 5'b00100,  // Bitwise AND
    OP_OR       = 5'b00101,  // Bitwise OR
    OP_XOR      = 5'b00110,  // Bitwise XOR
    OP_NAND     = 5'b00111,  // Bitwise NAND

    // Shift Operations (010xx)
    OP_SLL      = 5'b01000,  // Shift left logical
    OP_SRL      = 5'b01001,  // Shift right logical
    OP_SRA      = 5'b01010,  // Shift right arithmetic
    OP_ROR      = 5'b01011,  // Rotate right

    // Comparison Operations (011xx)
    OP_SLT      = 5'b01100,  // Set less than (signed)
    OP_SLTU     = 5'b01101,  // Set less than (unsigned)
    OP_SGT      = 5'b01110,  // Set greater than
```

alu_pkg.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/Al

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | alu_pipelined.sv | **alu_pkg.sv** | alu_writeback_stage.sv | arith_unit.sv

```
  // Comparison Operations (011xx)
  OP_SLT      = 5'b01100,  // Set less than (signed)
  OP_SLTU     = 5'b01101,  // Set less than (unsigned)
  OP_SGT      = 5'b01110,  // Set greater than
  OP_SEQ      = 5'b01111,  // Set equal

  // Multiply Operations (100xx)
  OP_MUL      = 5'b10000,  // Multiply (lower 32 bits)
  OP_MULH     = 5'b10001,  // Multiply high (signed)
  OP_MULHU    = 5'b10010,  // Multiply high (unsigned)
  OP_MULSU    = 5'b10011,  // Multiply (signed x unsigned)

  // Divide Operations (101xx)
  OP_DIV      = 5'b10100,  // Signed division
  OP_DIVU     = 5'b10101,  // Unsigned division
  OP_REM      = 5'b10110,  // Signed remainder
  OP_REMU     = 5'b10111,  // Unsigned remainder

  // Saturation Operations (110xx)
  OP_SAT_ADD  = 5'b11000,  // Saturating addition
  OP_SAT_SUB  = 5'b11001,  // Saturating subtraction
  OP_SAT_MUL  = 5'b11010,  // Saturating multiply
  OP_SAT_RSHIFT = 5'b11011, // Saturating right shift

  // Special Operations (111xx)
  OP_MIN      = 5'b11100,  // Minimum of A and B
  OP_MAX      = 5'b11101,  // Maximum of A and B
  OP_ABS      = 5'b11110,  // Absolute value of A
  OP_CLZ      = 5'b11111   // Count leading zeros
} opcode_t;

//==============================================================================
// Functional Unit Select (decoded from opcode[4:2])
//==============================================================================
typedef enum logic [2:0] {
  FUNC_ARITH    = 3'b000,
  FUNC_LOGIC    = 3'b001,
  FUNC_SHIFT    = 3'b010,
  FUNC_COMPARE  = 3'b011,
  FUNC_MULTIPLY = 3'b100,
  FUNC_DIVIDE   = 3'b101,
  FUNC_SATURATE = 3'b110,
  FUNC_SPECIAL  = 3'b111
} func_unit_t;

//==============================================================================
// Flag Bit Positions
//==============================================================================
typedef enum int {
```

alu_pkg.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (5

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | alu_pipelined.sv | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | comp

```
//============================================================================
// Flag Bit Positions
//============================================================================
typedef enum int {
  FLAG_SATURATION = 0,
  FLAG_PARITY     = 1,
  FLAG_NEGATIVE   = 2,
  FLAG_OVERFLOW   = 3,
  FLAG_CARRY      = 4,
  FLAG_ZERO       = 5
} flag_pos_t;

//============================================================================
// Pipeline Register Structures
//============================================================================

// Stage 1 to Stage 2 register
typedef struct packed {
  logic [DATA_WIDTH-1:0]    operand_a;
  logic [DATA_WIDTH-1:0]    operand_b;
  logic [OPCODE_WIDTH-1:0]  opcode;
  logic                     carry_in;   // For ADDC/SUBB
  logic [REG_ADDR_WIDTH-1:0] src_a_addr; // Source register A address
  logic [REG_ADDR_WIDTH-1:0] src_b_addr; // Source register B address
  logic [REG_ADDR_WIDTH-1:0] dest_addr;  // Destination register address
  logic                     valid;
} stage1_reg_t;

// Stage 2 to Stage 3 register
typedef struct packed {
  logic [DATA_WIDTH-1:0]    result;
  logic [FLAG_WIDTH-1:0]    flags;
  logic [REG_ADDR_WIDTH-1:0] dest_addr;  // Destination register address
  logic                     valid;
} stage2_reg_t;

//============================================================================
// Forwarding Select
//============================================================================
typedef enum logic [1:0] {
  FWD_NONE   = 2'b00,  // Use register value (no hazard)
  FWD_STAGE2 = 2'b01,  // Forward from Stage 2
  FWD_STAGE3 = 2'b10   // Forward from Stage 3
} fwd_sel_t;

//============================================================================
// Helper Functions
//============================================================================
```

alu_pkg.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (5 of 18)

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | alu_pipelined.sv | **alu_pkg.sv** | alu_writeback_stage.sv | arith_unit.sv | compare_unit

```
  //==============================================================================
  // Forwarding Select
  //==============================================================================
  typedef enum logic [1:0] {
    FWD_NONE   = 2'b00,  // Use register value (no hazard)
    FWD_STAGE2 = 2'b01,  // Forward from Stage 2
    FWD_STAGE3 = 2'b10   // Forward from Stage 3
  } fwd_sel_t;

  //==============================================================================
  // Helper Functions
  //==============================================================================

  // Get functional unit from opcode
  function automatic func_unit_t get_func_unit(input logic [OPCODE_WIDTH-1:0] opcode);
    return func_unit_t'(opcode[4:2]);
  endfunction

  // Check if operation is arithmetic (for carry/overflow flags)
  function automatic logic is_arithmetic_op(input logic [OPCODE_WIDTH-1:0] opcode);
    func_unit_t fu = get_func_unit(opcode);
    return (fu == FUNC_ARITH);
  endfunction

  // Check if operation is saturation type
  function automatic logic is_saturation_op(input logic [OPCODE_WIDTH-1:0] opcode);
    func_unit_t fu = get_func_unit(opcode);
    return (fu == FUNC_SATURATE);
  endfunction

  // Count leading zeros function
  function automatic logic [5:0] count_leading_zeros(input logic [DATA_WIDTH-1:0] value);
    logic [5:0] count;
    count = 0;
    for (int i = DATA_WIDTH-1; i >= 0; i--) begin
      if (value[i] == 1'b1)
        return count;
      count++;
    end
    return DATA_WIDTH; // All zeros
  endfunction

endpackage : alu_pkg
~
~
~
~
```

alu_core.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/U

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | alu_pipelined.sv | alu_pkg.sv | alu_writeback_stage.sv | arith_

```
//------------------------------------------------------------------------------
// Module: alu_core
// Description: ALU Core - Instantiates all functional units and multiplexes results
//------------------------------------------------------------------------------

module alu_core
  import alu_pkg::*;
#(
  parameter int DATA_WIDTH = 32
)(
  // Operands
  input  logic [DATA_WIDTH-1:0]   operand_a,
  input  logic [DATA_WIDTH-1:0]   operand_b,

  // Opcode
  input  logic [OPCODE_WIDTH-1:0] opcode,

  // Carry input for ADDC/SUBB
  input  logic                    carry_in,

  // Results
  output logic [DATA_WIDTH-1:0]   result,
  output logic                    carry_out,
  output logic                    overflow_out,
  output logic                    saturation_out
);

  //============================================================================
  // Internal Signals
  //============================================================================

  // Functional unit select
  logic [2:0] func_sel;
  logic [1:0] op_sel;

  // Results from each functional unit
  logic [DATA_WIDTH-1:0] arith_result;
  logic [DATA_WIDTH-1:0] logic_result;
  logic [DATA_WIDTH-1:0] shift_result;
  logic [DATA_WIDTH-1:0] compare_result;
  logic [DATA_WIDTH-1:0] multiply_result;
  logic [DATA_WIDTH-1:0] divide_result;
  logic [DATA_WIDTH-1:0] saturation_result;
  logic [DATA_WIDTH-1:0] special_result;

  // Arithmetic unit flags
  logic arith_carry;
  logic arith_overflow;

  // Saturation unit flag
```

alu_core.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | alu_pipelined.sv | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv

```
// Arithmetic unit flags
logic arith_carry;
logic arith_overflow;

// Saturation unit flag
logic sat_occurred;

//============================================================================
// Opcode Decode
//============================================================================
assign func_sel = opcode[4:2];  // Functional unit select
assign op_sel   = opcode[1:0];  // Operation within unit

//============================================================================
// Functional Unit Instantiations
//============================================================================

// Arithmetic Unit (ADD, SUB, ADDC, SUBB)
arith_unit #(
  .DATA_WIDTH(DATA_WIDTH)
) u_arith_unit (
  .operand_a (operand_a),
  .operand_b (operand_b),
  .op_sel    (op_sel),
  .carry_in  (carry_in),
  .result    (arith_result),
  .carry_out (arith_carry),
  .overflow  (arith_overflow)
);

// Logic Unit (AND, OR, XOR, NAND)
logic_unit #(
  .DATA_WIDTH(DATA_WIDTH)
) u_logic_unit (
  .operand_a (operand_a),
  .operand_b (operand_b),
  .op_sel    (op_sel),
  .result    (logic_result)
);

// Shift Unit (SLL, SRL, SRA, ROR)
shift_unit #(
  .DATA_WIDTH(DATA_WIDTH)
) u_shift_unit (
  .operand_a (operand_a),
  .operand_b (operand_b),
  .op_sel    (op_sel),
  .result    (shift_result)
);
```

alu_core.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | alu_pipelined.sv | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | co

```
);

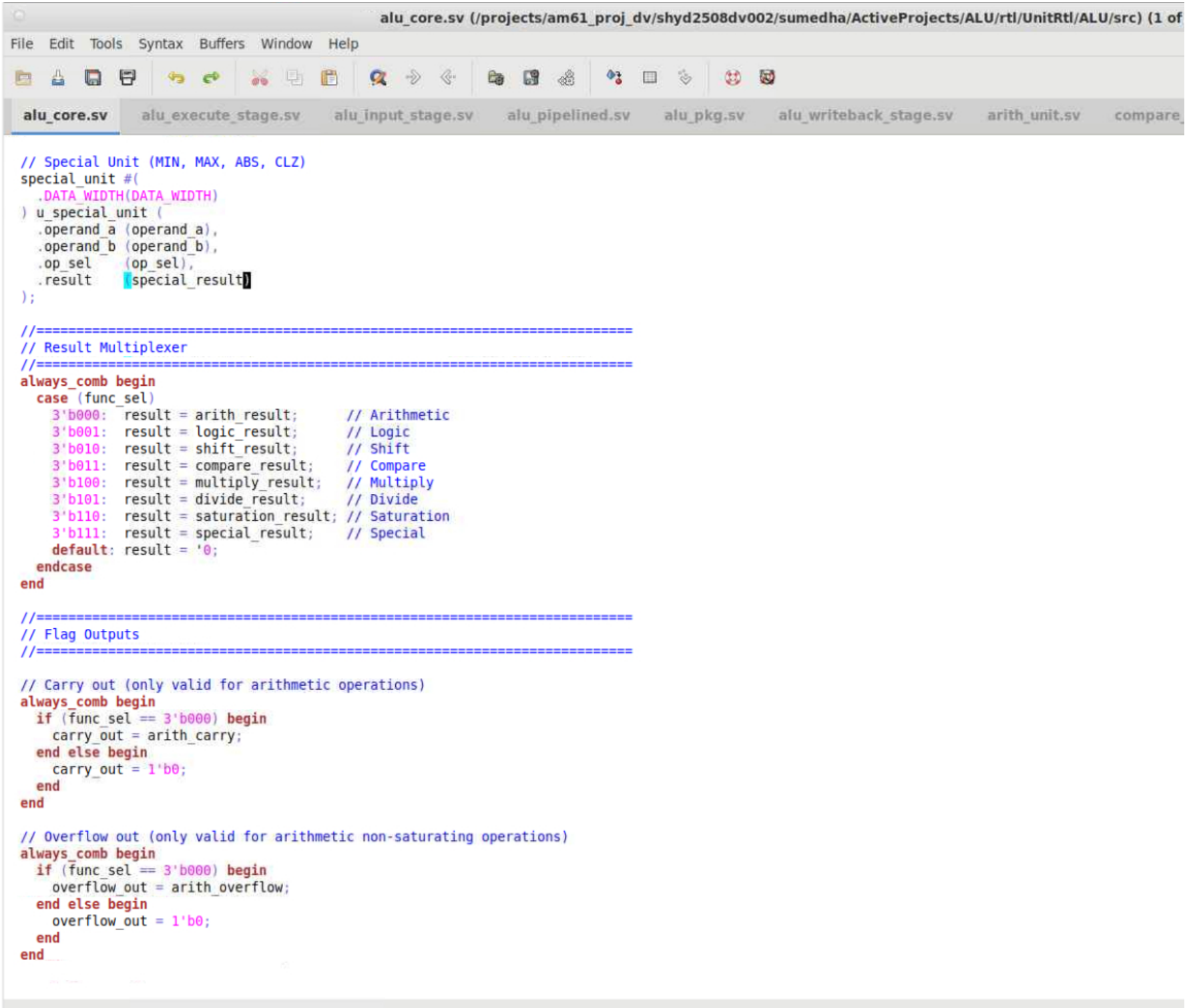
// Compare Unit (SLT, SLTU, SGT, SEQ)
compare_unit #(
  .DATA_WIDTH(DATA_WIDTH)
) u_compare_unit (
  .operand_a (operand_a),
  .operand_b (operand_b),
  .op_sel    (op_sel),
  .result    (compare_result)
);

// Multiply Unit (MUL, MULH, MULHU, MULSU)
multiply_unit #(
  .DATA_WIDTH(DATA_WIDTH)
) u_multiply_unit (
  .operand_a (operand_a),
  .operand_b (operand_b),
  .op_sel    (op_sel),
  .result    (multiply_result)
);

// Divide Unit (DIV, DIVU, REM, REMU)
divide_unit #(
  .DATA_WIDTH(DATA_WIDTH)
) u_divide_unit (
  .operand_a (operand_a),
  .operand_b (operand_b),
  .op_sel    (op_sel),
  .result    (divide_result)
);

// Saturation Unit (SAT_ADD, SAT_SUB, SAT_MUL, SAT_RSHIFT)
saturation_unit #(
  .DATA_WIDTH(DATA_WIDTH)
) u_saturation_unit (
  .operand_a          (operand_a),
  .operand_b          (operand_b),
  .op_sel             (op_sel),
  .result             (saturation_result),
  .saturation_occurred (sat_occurred)
);

// Special Unit (MIN, MAX, ABS, CLZ)
special_unit #(
  .DATA_WIDTH(DATA_WIDTH)
) u_special_unit (
  .operand_a (operand_a),
  .operand_b (operand_b),
  .op_sel    (op_sel),
```


alu_core.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (1 of
File Edit Tools Syntax Buffers Window Help
alu_core.sv
alu_execute_stage.sv
alu_input_stage.sv
alu_pipelined.sv
alu_pkg.sv
alu_writeback_stage.sv
arith_unit.sv
compare_
// Special Unit (MIN, MAX, ABS, CLZ)
special_unit #(
.DATA_WIDTH(DATA_WIDTH)
) u_special_unit (
.operand_a (operand_a),
.operand_b (operand_b),
.op_sel (op_sel),
.result (special_result)
);
//==========
// Result Multiplexer
//==========
always_comb begin
case (func_sel)
3'b000: result = arith_result; // Arithmetic
3'b001: result = logic_result; // Logic
3'b010: result = shift_result; // Shift
3'b011: result = compare_result; // Compare
3'b100: result = multiply_result; // Multiply
3'b101: result = divide_result; // Divide
3'b110: result = saturation_result; // Saturation
3'b111: result = special_result; // Special
default: result = '0;
endcase
end
//==========
// Flag Outputs
//==========
// Carry out (only valid for arithmetic operations)
always_comb begin
if (func_sel == 3'b000) begin
carry_out = arith_carry;
end else begin
carry_out = 1'b0;
end
end
// Overflow out (only valid for arithmetic non-saturating operations)
always_comb begin
if (func_sel == 3'b000) begin
overflow_out = arith_overflow;
end else begin
overflow_out = 1'b0;
end
end

alu_core.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (1 of 18) - GVIM9

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | alu_pipelined.sv | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | compare_unit.sv | d

```
      3'b001:  result = logic_result;      // Logic
      3'b010:  result = shift_result;      // Shift
      3'b011:  result = compare_result;    // Compare
      3'b100:  result = multiply_result;   // Multiply
      3'b101:  result = divide_result;     // Divide
      3'b110:  result = saturation_result; // Saturation
      3'b111:  result = special_result;    // Special
      default: result = '0;
    endcase
  end

  //============================================================================
  // Flag Outputs
  //============================================================================

  // Carry out (only valid for arithmetic operations)
  always_comb begin
    if (func_sel == 3'b000) begin
      carry_out = arith_carry;
    end else begin
      carry_out = 1'b0;
    end
  end

  // Overflow out (only valid for arithmetic non-saturating operations)
  always_comb begin
    if (func_sel == 3'b000) begin
      overflow_out = arith_overflow;
    end else begin
      overflow_out = 1'b0;
    end
  end

  // Saturation out (only valid for saturation operations)
  always_comb begin
    if (func_sel == 3'b110) begin
      saturation_out = sat_occurred;
    end else begin
      saturation_out = 1'b0;
    end
  end

endmodule : alu_core
~
~
~
~
~
```

arith_unit.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (7 of 18) - GV

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | alu_pipelined.sv | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | compare_unit.sv

```
//------------------------------------------------------------------------------
// Module: arith_unit
// Description: Arithmetic Unit for ALU
//              Implements ADD, SUB, ADDC, SUBB operations
//------------------------------------------------------------------------------

module arith_unit
  import alu_pkg::*;
#(
  parameter int DATA_WIDTH = 32
)(
  // Operands
  input  logic [DATA_WIDTH-1:0] operand_a,
  input  logic [DATA_WIDTH-1:0] operand_b,

  // Operation select (opcode[1:0])
  input  logic [1:0]            op_sel,

  // Carry input for ADDC/SUBB
  input  logic                  carry_in,

  // Results
  output logic [DATA_WIDTH-1:0] result,
  output logic                  carry_out,
  output logic                  overflow
);

  //============================================================================
  // Internal Signals
  //============================================================================
  logic [DATA_WIDTH:0] result_ext;  // 33-bit for carry detection
  logic [DATA_WIDTH:0] a_ext;
  logic [DATA_WIDTH:0] b_ext;
  logic                cin;
  logic                is_subtract;

  //============================================================================
  // Operation Logic
  //============================================================================
  always_comb begin
    // Extend operands to 33 bits
    a_ext = {1'b0, operand_a};

    // Determine operation type
    case (op_sel)
      2'b00: begin // ADD
        b_ext       = {1'b0, operand_b};
        cin         = 1'b0;
        is_subtract = 1'b0;
      end
```

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | alu_pipelined.sv | alu_pkg.sv | alu_writeback_stage.sv | **arith_unit.sv** | compare_unit.sv

```
    2'b00: begin // ADD
      b_ext       = {1'b0, operand_b};
      cin         = 1'b0;
      is_subtract = 1'b0;
    end

    2'b01: begin // SUB: A - B = A + (~B) + 1
      b_ext       = {1'b0, ~operand_b};
      cin         = 1'b1;
      is_subtract = 1'b1;
    end

    2'b10: begin // ADDC: A + B + Cin
      b_ext       = {1'b0, operand_b};
      cin         = carry_in;
      is_subtract = 1'b0;
    end

    2'b11: begin // SUBB: A - B - ~Cin = A + (~B) + Cin
      b_ext       = {1'b0, ~operand_b};
      cin         = carry_in;
      is_subtract = 1'b1;
    end

    default: begin
      b_ext       = '0;
      cin         = 1'b0;
      is_subtract = 1'b0;
    end
  endcase

  // Perform addition
  result_ext = a_ext + b_ext + {{DATA_WIDTH{1'b0}}, cin};

  // Extract result and carry
  result    = result_ext[DATA_WIDTH-1:0];
  carry_out = result_ext[DATA_WIDTH];

  // Overflow detection for signed arithmetic
  // ADD/ADDC: overflow when same sign inputs produce different sign output
  // SUB/SUBB: overflow when different sign inputs produce unexpected sign
  if (!is_subtract) begin
    // Addition overflow: pos + pos = neg, or neg + neg = pos
    overflow = (operand_a[DATA_WIDTH-1] == operand_b[DATA_WIDTH-1]) &&
               (operand_a[DATA_WIDTH-1] != result[DATA_WIDTH-1]);
  end else begin
    // Subtraction overflow: pos - neg = neg, or neg - pos = pos
    overflow = (operand_a[DATA_WIDTH-1] != operand_b[DATA_WIDTH-1]) &&
               (operand_a[DATA_WIDTH-1] != result[DATA_WIDTH-1]);
  end
```

compare_unit.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (

File Edit Tools Syntax Buffers Window Help

alu_core.sv alu_execute_stage.sv alu_input_stage.sv alu_pipelined.sv alu_pkg.sv alu_writeback_stage.sv arith_unit.sv compare

```
//------------------------------------------------------------------------------
// Module: compare_unit
// Description: Comparison Unit for ALU
//              Implements SLT, SLTU, SGT, SEQ operations
//------------------------------------------------------------------------------

module compare_unit
  import alu_pkg::*;
#(
  parameter int DATA_WIDTH = 32
)(
  // Operands
  input  logic [DATA_WIDTH-1:0] operand_a,
  input  logic [DATA_WIDTH-1:0] operand_b,

  // Operation select (opcode[1:0])
  input  logic [1:0]            op_sel,

  // Result (1 or 0)
  output logic [DATA_WIDTH-1:0] result
);

  //============================================================================
  // Internal Signals
  //============================================================================
  logic signed [DATA_WIDTH-1:0] signed_a;
  logic signed [DATA_WIDTH-1:0] signed_b;

  //============================================================================
  // Signed Conversion
  //============================================================================
  assign signed_a = $signed(operand_a);
  assign signed_b = $signed(operand_b);

  //============================================================================
  // Operation Logic
  //============================================================================
  always_comb begin
    case (op_sel)
      2'b00: begin // SLT - Set Less Than (Signed)
        result = (signed_a < signed_b) ? {{(DATA_WIDTH-1){1'b0}}, 1'b1} : '0;
      end

      2'b01: begin // SLTU - Set Less Than Unsigned
        result = (operand_a < operand_b) ? {{(DATA_WIDTH-1){1'b0}}, 1'b1} : '0;
      end

      2'b10: begin // SGT - Set Greater Than (Signed)
        result = (signed_a > signed_b) ? {{(DATA_WIDTH-1){1'b0}}, 1'b1} : '0;
      end
```

compare_unit.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/AL

File Edit Tools Syntax Buffers Window Help

alu_core.sv alu_execute_stage.sv alu_input_stage.sv alu_pipelined.sv alu_pkg.sv alu_writeback_stage.sv arith_unit.sv

```
  // Result (1 or 0)
  output logic [DATA_WIDTH-1:0] result
);

  //============================================================================
  // Internal Signals
  //============================================================================
  logic signed [DATA_WIDTH-1:0] signed_a;
  logic signed [DATA_WIDTH-1:0] signed_b;

  //============================================================================
  // Signed Conversion
  //============================================================================
  assign signed_a = $signed(operand_a);
  assign signed_b = $signed(operand_b);

  //============================================================================
  // Operation Logic
  //============================================================================
  always_comb begin
    case (op_sel)
      2'b00: begin // SLT - Set Less Than (Signed)
        result = (signed_a < signed_b) ? {{(DATA_WIDTH-1){1'b0}}, 1'b1} : '0;
      end

      2'b01: begin // SLTU - Set Less Than Unsigned
        result = (operand_a < operand_b) ? {{(DATA_WIDTH-1){1'b0}}, 1'b1} : '0;
      end

      2'b10: begin // SGT - Set Greater Than (Signed)
        result = (signed_a > signed_b) ? {{(DATA_WIDTH-1){1'b0}}, 1'b1} : '0;
      end

      2'b11: begin // SEQ - Set Equal
        result = (operand_a == operand_b) ? {{(DATA_WIDTH-1){1'b0}}, 1'b1} : '0;
      end

      default: result = '0;
    endcase
  end

endmodule : compare_unit
~
~
~
~
~
```

divide_unit.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (9 of 18) - GVIM9

File Edit Tools Syntax Buffers Window Help

alu_core.sv alu_execute_stage.sv alu_input_stage.sv alu_pipelined.sv alu_pkg.sv alu_writeback_stage.sv arith_unit.sv compare_unit.sv d

```
//------------------------------------------------------------------------------
// Module: divide_unit
// Description: Divide Unit for ALU
//              Implements DIV, DIVU, REM, REMU operations
//------------------------------------------------------------------------------

module divide_unit
  import alu_pkg::*;
#(
  parameter int DATA_WIDTH = 32
)(
  // Operands
  input  logic [DATA_WIDTH-1:0] operand_a,  // Dividend
  input  logic [DATA_WIDTH-1:0] operand_b,  // Divisor

  // Operation select (opcode[1:0])
  input  logic [1:0]            op_sel,

  // Result
  output logic [DATA_WIDTH-1:0] result
);

  //============================================================================
  // Internal Signals
  //============================================================================
  logic signed [DATA_WIDTH-1:0] signed_a;
  logic signed [DATA_WIDTH-1:0] signed_b;

  // Division results
  logic [DATA_WIDTH-1:0] quotient_signed;
  logic [DATA_WIDTH-1:0] quotient_unsigned;
  logic [DATA_WIDTH-1:0] remainder_signed;
  logic [DATA_WIDTH-1:0] remainder_unsigned;

  // Divide by zero flag
  logic div_by_zero;

  //============================================================================
  // Signed Conversion
  //============================================================================
  assign signed_a = $signed(operand_a);
  assign signed_b = $signed(operand_b);

  assign div_by_zero = (operand_b == '0);

  //============================================================================
  // Division Operations
  //============================================================================
  always_comb begin
    if (div_by_zero) begin
```

divide_unit.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (9 of 18)

File Edit Tools Syntax Buffers Window Help

alu_core.sv alu_execute_stage.sv alu_input_stage.sv alu_pipelined.sv alu_pkg.sv alu_writeback_stage.sv arith_unit.sv compare_unit.

```
  assign div_by_zero = (operand_b == '0);

  //============================================================================
  // Division Operations
  //============================================================================
  always_comb begin
    if (div_by_zero) begin
      // Divide by zero handling
      // Quotient returns all 1s, remainder returns dividend
      quotient_signed   = '1;
      quotient_unsigned = '1;
      remainder_signed  = operand_a;
      remainder_unsigned = operand_a;
    end else begin
      // Normal division
      quotient_signed    = signed_a / signed_b;
      quotient_unsigned  = operand_a / operand_b;
      remainder_signed   = signed_a % signed_b;
      remainder_unsigned = operand_a % operand_b;
    end
  end

  //============================================================================
  // Operation Logic
  //============================================================================
  always_comb begin
    case (op_sel)
      2'b00: begin // DIV - Signed Division
        result = quotient_signed;
      end

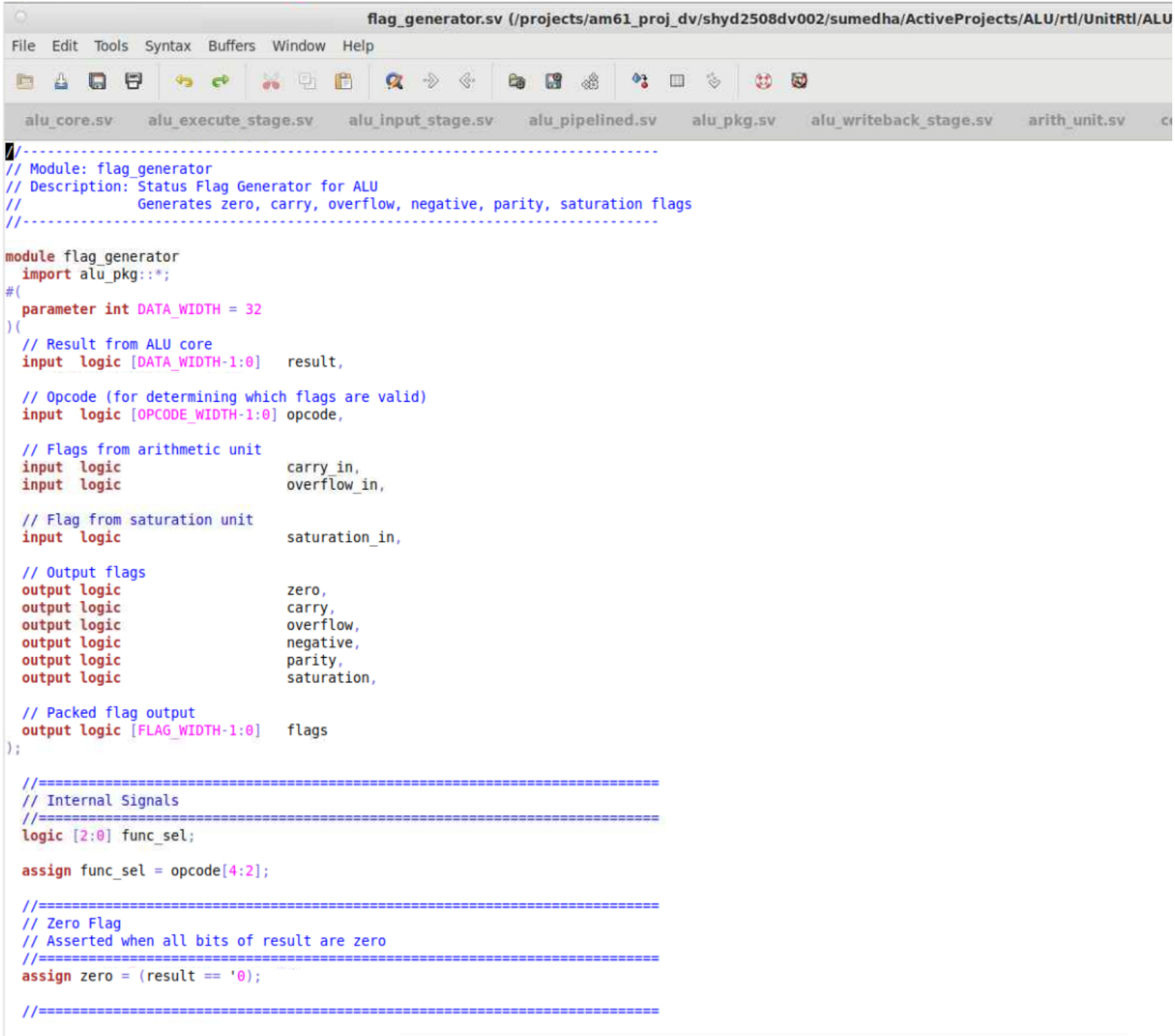
      2'b01: begin // DIVU - Unsigned Division
        result = quotient_unsigned;
      end

      2'b10: begin // REM - Signed Remainder
        result = remainder_signed;
      end

      2'b11: begin // REMU - Unsigned Remainder
        result = remainder_unsigned;
      end

      default: result = '0;
    endcase
  end

endmodule : divide_unit
~
```


flag_generator.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU
File Edit Tools Syntax Buffers Window Help
alu_core.sv alu_execute_stage.sv alu_input_stage.sv alu_pipelined.sv alu_pkg.sv alu_writeback_stage.sv arith_unit.sv
//--
// Module: flag_generator
// Description: Status Flag Generator for ALU
//   Generates zero, carry, overflow, negative, parity, saturation flags
//--
module flag_generator
import alu_pkg::*;
#(
parameter int DATA_WIDTH = 32
)(
// Result from ALU core
input logic [DATA_WIDTH-1:0] result,
// Opcode (for determining which flags are valid)
input logic [OPCODE_WIDTH-1:0] opcode,
// Flags from arithmetic unit
input logic carry_in,
input logic overflow_in,
// Flag from saturation unit
input logic saturation_in,
// Output flags
output logic zero,
output logic carry,
output logic overflow,
output logic negative,
output logic parity,
output logic saturation,
// Packed flag output
output logic [FLAG_WIDTH-1:0] flags
);
//==
// Internal Signals
//==
logic [2:0] func_sel;
assign func_sel = opcode[4:2];
//==
// Zero Flag
// Asserted when all bits of result are zero
//==
assign zero = (result == '0);
//==

flag_generator.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (

File Edit Tools Syntax Buffers Window Help

alu_core.sv alu_execute_stage.sv alu_input_stage.sv alu_pipelined.sv alu_pkg.sv alu_writeback_stage.sv arith_unit.sv compar

```
// Asserted when all bits of result are zero
//============================================================================
assign zero = (result == '0);

//============================================================================
// Negative Flag
// Reflects the MSB (sign bit) of the result
//============================================================================
assign negative = result[DATA_WIDTH-1];

//============================================================================
// Parity Flag
// Even parity - XOR reduction of all result bits
// 0 = even number of 1s, 1 = odd number of 1s
//============================================================================
assign parity = ^result;

//============================================================================
// Carry Flag
// Only valid for arithmetic operations (func_sel = 000)
//============================================================================
always_comb begin
  if (func_sel == 3'b000) begin
    carry = carry_in;
  end else begin
    carry = 1'b0;
  end
end

//============================================================================
// Overflow Flag
// Only valid for arithmetic operations (func_sel = 000)
// Never set for saturation operations
//============================================================================
always_comb begin
  if (func_sel == 3'b000) begin
    overflow = overflow_in;
  end else begin
    overflow = 1'b0;
  end
end

//============================================================================
// Saturation Flag
// Only valid for saturation operations (func_sel = 110)
//============================================================================
always_comb begin
  if (func_sel == 3'b110) begin
    saturation = saturation_in;
  end else begin
```

Applications Places System

flag_generator.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (10 of 18) - GVIM

File Edit Tools Syntax Buffers Window Help

alu_core.sv alu_execute_stage.sv alu_input_stage.sv alu_pipelined.sv alu_pkg.sv alu_writeback_stage.sv arith_unit.sv compare_unit.sv div

```
    end else begin
      carry = 1'b0;
    end
  end

  //============================================================================
  // Overflow Flag
  // Only valid for arithmetic operations (func_sel = 000)
  // Never set for saturation operations
  //============================================================================
  always_comb begin
    if (func_sel == 3'b000) begin
      overflow = overflow_in;
    end else begin
      overflow = 1'b0;
    end
  end

  //============================================================================
  // Saturation Flag
  // Only valid for saturation operations (func_sel = 110)
  //============================================================================
  always_comb begin
    if (func_sel == 3'b110) begin
      saturation = saturation_in;
    end else begin
      saturation = 1'b0;
    end
  end

  //============================================================================
  // Packed Flag Output
  // Order: {zero, carry, overflow, negative, parity, saturation}
  //============================================================================
  assign flags = {zero, carry, overflow, negative, parity, saturation};

endmodule : flag_generator
~
~
~
~
~
```

alu_input_stage.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (3 of 18) - GVIM9 (on ip

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | alu_pipelined.sv | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | compare_unit.sv | divide_un

```
//------------------------------------------------------------------------------
// Module: alu_input_stage
// Description: Pipeline Stage 1 - Input/Fetch/Decode Stage
//              Captures input operands and opcode, applies forwarding,
//              and registers data for the execute stage
//------------------------------------------------------------------------------

module alu_input_stage
  import alu_pkg::*;
#(
  parameter int DATA_WIDTH = 32
)(
  // Clock and Reset
  input  logic                    clk,
  input  logic                    rst_n,

  // Input Interface (Valid/Ready Handshake)
  input  logic                    in_valid,
  output logic                    in_ready,

  // Input Operands and Control
  input  logic [DATA_WIDTH-1:0]     operand_a,
  input  logic [DATA_WIDTH-1:0]     operand_b,
  input  logic [OPCODE_WIDTH-1:0]   opcode,
  input  logic                      carry_in,

  // Register Addresses (for hazard detection)
  input  logic [REG_ADDR_WIDTH-1:0] src_a_addr,
  input  logic [REG_ADDR_WIDTH-1:0] src_b_addr,
  input  logic [REG_ADDR_WIDTH-1:0] dest_addr,

  // Forwarded values from later pipeline stages
  input  logic [DATA_WIDTH-1:0]   stage2_result,
  input  logic [DATA_WIDTH-1:0]   stage3_result,

  // Forwarding select signals from hazard detection
  input  fwd_sel_t                forward_a_sel,
  input  fwd_sel_t                forward_b_sel,

  // Pipeline stall control
  input  logic                    stall,

  // Output to Execute Stage (Stage 1 Register)
  output stage1_reg_t             stage1_out
);

  //============================================================================
  // Internal Signals
  //============================================================================
  logic [DATA_WIDTH-1:0] operand_a_fwd;
```

alu_input_stage.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/sr

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | alu_pipelined.sv | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | comp

```
//============================================================================
// Internal Signals
//============================================================================
logic [DATA_WIDTH-1:0] operand_a_fwd;
logic [DATA_WIDTH-1:0] operand_b_fwd;
logic                  accept_input;

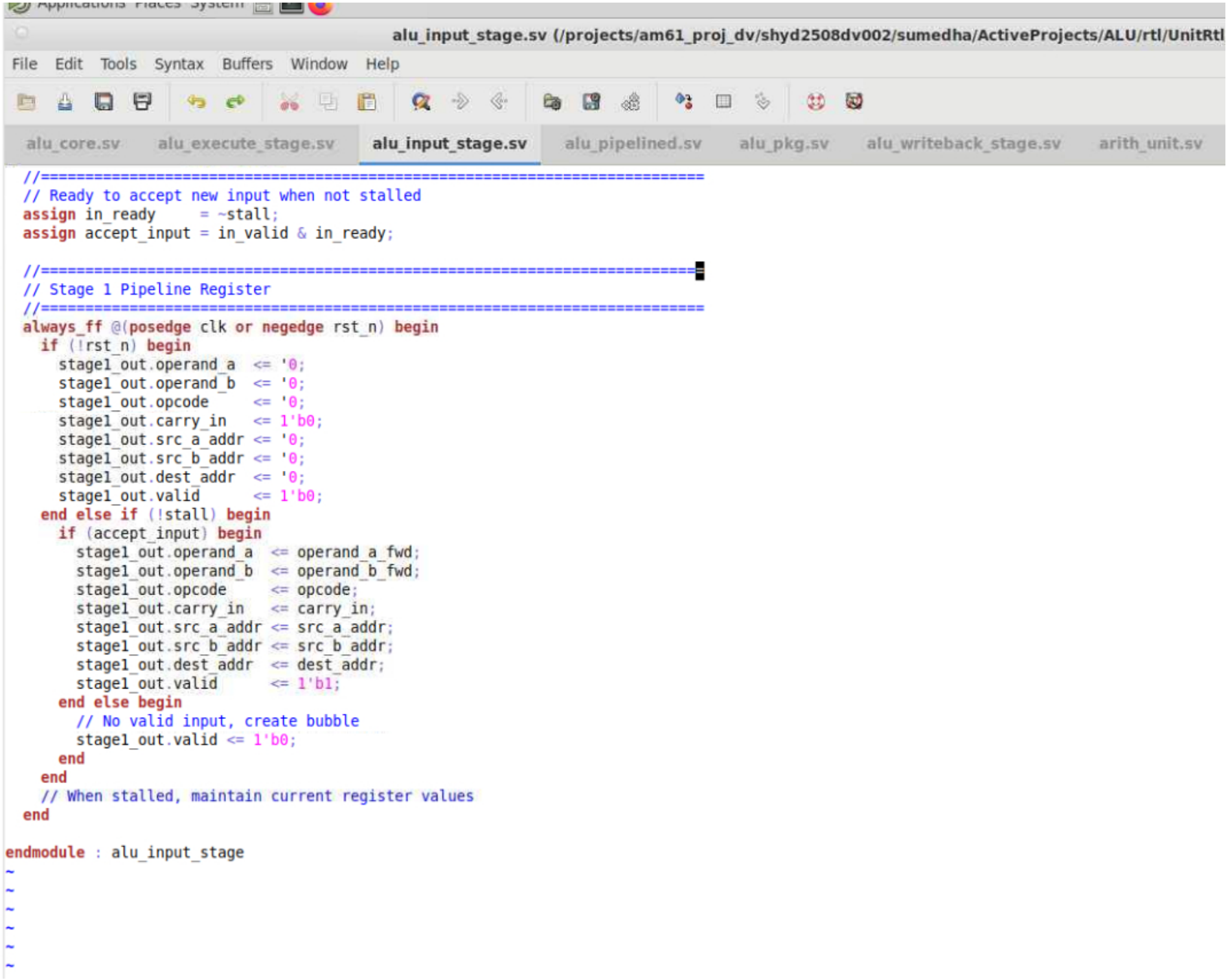
//============================================================================
// Forwarding Multiplexers
//============================================================================

// Forwarding MUX for Operand A
forwarding_mux #(
  .DATA_WIDTH(DATA_WIDTH)
) u_fwd_mux_a (
  .operand_in    (operand_a),
  .stage2_result (stage2_result),
  .stage3_result (stage3_result),
  .fwd_sel       (forward_a_sel),
  .operand_out   (operand_a_fwd)
);

// Forwarding MUX for Operand B
forwarding_mux #(
  .DATA_WIDTH(DATA_WIDTH)
) u_fwd_mux_b (
  .operand_in    (operand_b),
  .stage2_result (stage2_result),
  .stage3_result (stage3_result),
  .fwd_sel       (forward_b_sel),
  .operand_out   (operand_b_fwd)
);

//============================================================================
// Ready/Accept Logic
//============================================================================
// Ready to accept new input when not stalled
assign in_ready     = ~stall;
assign accept_input = in_valid & in_ready;

//============================================================================
// Stage 1 Pipeline Register
//============================================================================
always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    stage1_out.operand_a  <= '0;
    stage1_out.operand_b  <= '0;
    stage1_out.opcode     <= '0;
    stage1_out.carry_in   <= 1'b0;
```


alu_input_stage.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl
File Edit Tools Syntax Buffers Window Help
alu_core.sv
alu_execute_stage.sv
alu_input_stage.sv
alu_pipelined.sv
alu_pkg.sv
alu_writeback_stage.sv
arith_unit.sv
//==========
// Ready to accept new input when not stalled
assign in_ready = ~stall;
assign accept_input = in_valid & in_ready;
//==========
// Stage 1 Pipeline Register
//==========
always_ff @(posedge clk or negedge rst_n) begin
if (!rst_n) begin
stage1_out.operand_a <= '0;
stage1_out.operand_b <= '0;
stage1_out.opcode <= '0;
stage1_out.carry_in <= 1'b0;
stage1_out.src_a_addr <= '0;
stage1_out.src_b_addr <= '0;
stage1_out.dest_addr <= '0;
stage1_out.valid <= 1'b0;
end else if (!stall) begin
if (accept_input) begin
stage1_out.operand_a <= operand_a_fwd;
stage1_out.operand_b <= operand_b_fwd;
stage1_out.opcode <= opcode;
stage1_out.carry_in <= carry_in;
stage1_out.src_a_addr <= src_a_addr;
stage1_out.src_b_addr <= src_b_addr;
stage1_out.dest_addr <= dest_addr;
stage1_out.valid <= 1'b1;
end else begin
// No valid input, create bubble
stage1_out.valid <= 1'b0;
end
end
// When stalled, maintain current register values
end
endmodule : alu_input_stage

alu_execute_stage.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (2 of 1

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | alu_pipelined.sv | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | compare_unit.sv

```
//------------------------------------------------------------------------------
// Module: alu_execute_stage
// Description: Pipeline Stage 2 - Execute Stage
//              Performs ALU computation using alu_core and generates flags
//              using flag_generator, then registers results for writeback
//------------------------------------------------------------------------------

module alu_execute_stage
  import alu_pkg::*;
#(
  parameter int DATA_WIDTH = 32
)(
  // Clock and Reset
  input  logic         clk,
  input  logic         rst_n,

  // Input from Stage 1
  input  stage1_reg_t  stage1_in,

  // Pipeline stall control
  input  logic         stall,

  // Output to Writeback Stage (Stage 2 Register)
  output stage2_reg_t  stage2_out,

  // Combinational result output (for forwarding)
  output logic [DATA_WIDTH-1:0] result_comb
);

  //============================================================================
  // Internal Signals
  //============================================================================
  logic [DATA_WIDTH-1:0] alu_result;
  logic                  carry_out;
  logic                  overflow_out;
  logic                  saturation_out;

  // Individual flags
  logic                  zero_flag;
  logic                  carry_flag;
  logic                  overflow_flag;
  logic                  negative_flag;
  logic                  parity_flag;
  logic                  saturation_flag;
  logic [FLAG_WIDTH-1:0] flags_combined;

  //============================================================================
  // ALU Core Instantiation
  //============================================================================
  alu_core #(
```

alu_execute_stage.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (2 of 18) -

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | alu_pipelined.sv | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | compare_unit.sv

```
//==============================================================================
// ALU Core Instantiation
//==============================================================================
alu_core #(
  .DATA_WIDTH(DATA_WIDTH)
) u_alu_core (
  .operand_a      (stage1_in.operand_a),
  .operand_b      (stage1_in.operand_b),
  .opcode         (stage1_in.opcode),
  .carry_in       (stage1_in.carry_in),
  .result         (alu_result),
  .carry_out      (carry_out),
  .overflow_out   (overflow_out),
  .saturation_out (saturation_out)
);

//==============================================================================
// Flag Generator Instantiation
//==============================================================================
flag_generator #(
  .DATA_WIDTH(DATA_WIDTH)
) u_flag_generator (
  .result        (alu_result),
  .opcode        (stage1_in.opcode),
  .carry_in      (carry_out),
  .overflow_in   (overflow_out),
  .saturation_in (saturation_out),
  .zero          (zero_flag),
  .carry         (carry_flag),
  .overflow      (overflow_flag),
  .negative      (negative_flag),
  .parity        (parity_flag),
  .saturation    (saturation_flag),
  .flags         (flags_combined)
);

//==============================================================================
// Combinational Result Output (for forwarding)
//==============================================================================
assign result_comb = alu_result;

//==============================================================================
// Stage 2 Pipeline Register
//==============================================================================
always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    stage2_out.result    <= '0;
    stage2_out.flags     <= '0;
    stage2_out.dest_addr <= '0;
```

alu_execute_stage.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (2 of 18) -

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | alu_pipelined.sv | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | compare_unit.sv

```
    .overflow       (overflow_flag),
    .negative       (negative_flag),
    .parity         (parity_flag),
    .saturation     (saturation_flag),
    .flags          (flags_combined)
  );

  //============================================================================
  // Combinational Result Output (for forwarding)
  //============================================================================
  assign result_comb = alu_result;

  //============================================================================
  // Stage 2 Pipeline Register
  //============================================================================
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      stage2_out.result    <= '0;
      stage2_out.flags     <= '0;
      stage2_out.dest_addr <= '0;
      stage2_out.valid     <= 1'b0;
    end else if (!stall) begin
      if (stage1_in.valid) begin
        stage2_out.result    <= alu_result;
        stage2_out.flags     <= flags_combined;
        stage2_out.dest_addr <= stage1_in.dest_addr;
        stage2_out.valid     <= 1'b1;
      end else begin
        // No valid input from stage 1, create bubble
        stage2_out.valid <= 1'b0;
      end
    end
    // When stalled, maintain current register values
  end

endmodule : alu_execute_stage
~
~
~
~
~
~
~
```

alu_writeback_stage.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (6 o

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | alu_pipelined.sv | alu_pkg.sv | **alu_writeback_stage.sv** | arith_unit.sv | compare_unit.

```
//------------------------------------------------------------------------------
// Module: alu_writeback_stage
// Description: Pipeline Stage 3 - Writeback Stage
//              Provides final registered results and handles output handshake
//------------------------------------------------------------------------------

module alu_writeback_stage
  import alu_pkg::*;
#(
  parameter int DATA_WIDTH = 32
)(
  // Clock and Reset
  input  logic                    clk,
  input  logic                    rst_n,

  // Input from Stage 2
  input  stage2_reg_t             stage2_in,

  // Output Interface (Valid/Ready Handshake)
  output logic                    out_valid,
  input  logic                    out_ready,

  // Output Results
  output logic [DATA_WIDTH-1:0]     result,

  // Destination Address Output (for downstream hazard detection)
  output logic [REG_ADDR_WIDTH-1:0] dest_addr,

  // Individual Flag Outputs
  output logic                    zero,
  output logic                    carry,
  output logic                    overflow,
  output logic                    negative,
  output logic                    parity,
  output logic                    saturation,

  // Stage valid signal (for forwarding/hazard detection)
  output logic                    stage3_valid,

  // Pipeline stall signal (backpressure from output)
  output logic                    stall
);

  //============================================================================
  // Internal Signals
  //============================================================================
  logic [DATA_WIDTH-1:0]     result_reg;
  logic [FLAG_WIDTH-1:0]     flags_reg;
  logic [REG_ADDR_WIDTH-1:0] dest_addr_reg;
  logic                      valid_reg;
```

alu_writeback_stage.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/ALU/rtl/UnitRtl/ALU/src) (6 of 18)

File Edit Tools Syntax Buffers Window Help

alu_core.sv | alu_execute_stage.sv | alu_input_stage.sv | alu_pipelined.sv | alu_pkg.sv | alu_writeback_stage.sv | arith_unit.sv | compare_unit.sv

```
  //============================================================================
  // Stall Logic
  //============================================================================
  // Generate backpressure when output is valid but downstream is not ready
  assign stall = valid_reg & ~out_ready;

  //============================================================================
  // Stage 3 Pipeline Register (Output Register)
  //============================================================================
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      result_reg    <= '0;
      flags_reg     <= '0;
      dest_addr_reg <= '0;
      valid_reg     <= 1'b0;
    end else if (!stall) begin
      if (stage2_in.valid) begin
        result_reg    <= stage2_in.result;
        flags_reg     <= stage2_in.flags;
        dest_addr_reg <= stage2_in.dest_addr;
        valid_reg     <= 1'b1;
      end else begin
        // No valid input from stage 2, create bubble
        valid_reg <= 1'b0;
      end
    end
    // When stalled, maintain current register values
  end

  //============================================================================
  // Output Assignments
  //============================================================================
  assign out_valid    = valid_reg;
  assign result       = result_reg;
  assign dest_addr    = dest_addr_reg;
  assign stage3_valid = valid_reg;

  // Extract individual flags from packed format
  // Flag order: {zero, carry, overflow, negative, parity, saturation}
  assign zero       = flags_reg[FLAG_ZERO];
  assign carry      = flags_reg[FLAG_CARRY];
  assign overflow   = flags_reg[FLAG_OVERFLOW];
  assign negative   = flags_reg[FLAG_NEGATIVE];
  assign parity     = flags_reg[FLAG_PARITY];
  assign saturation = flags_reg[FLAG_SATURATION];

endmodule : alu_writeback_stage
~
~
~
```